Spring Cloud Config

主要内容

分布式配置中心介绍
基于 Gitee 管理配置文件
搭建分布式配置中心服务端
搭建分布式配置中心客户端
配置热刷新
分布式配置中心的安全认证

学习目标

知识点	要求
分布式配置中心介绍	了解
基于 Gitee 管理配置文件	掌握
搭建分布式配置中心服务端	掌握
搭建分布式配置中心客户端	掌握
配置热刷新	掌握
分布式配置中心的安全认证	掌握

Spring Cloud Config

一、 分布式配置中心介绍

1 什么是分布式配置中心

在常规的开发中，每个微服务都包含代码和配置。其配置包含服务配置、各类开关和业务配置。如果系统结构中的微服务节点较少，那么常规的代码+配置的开发方式足以解决问题。当系统逐步迭代，其微服务会越来越复杂，慢慢演化成网状依赖结构，这个时候常规的代码+配置的开发方式就并不合适了，因为还要考虑整体系统的扩展性、伸缩性和耦合性等。这些问题中，配置的管理也是非常麻烦的。

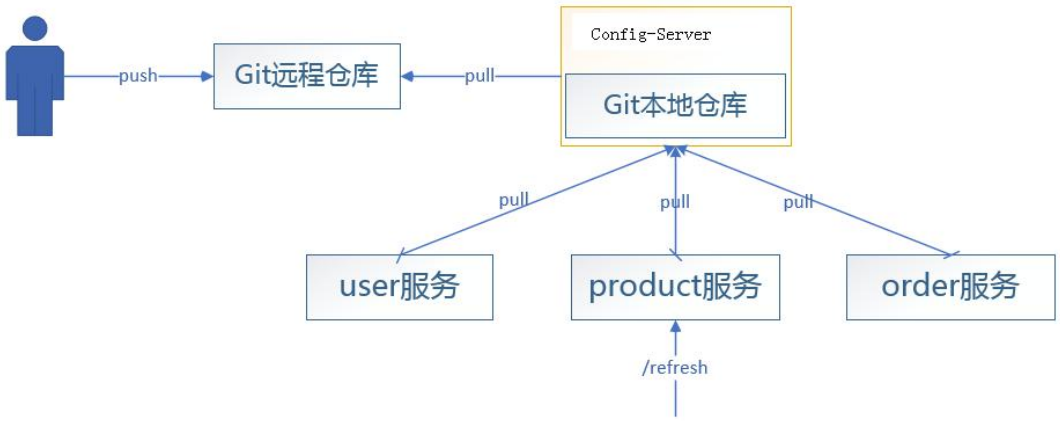
如果还是以常规开发形式管理配置，则要承担反复修改编译代码、重启系统、重新打包等风险。所以，一个可以集中管理，带有版本控制的配置中心应运而生。

spring cloud config 就是一个配置中心。其采用集中式管理每个微服务的配置信息，并使用 GIT 等版本仓库统一存储配置内容，实现版本化管理控制。微服务与配置中心使用 rest 方式交互来实现可扩展的配置服务。

spring cloud config 配置中心解决了微服务系统的配置中心化、配置版本控制、平台独立、语言独立等问题，其特性如下：

提供服务端和客户端支持（spring cloud config server 和 spring cloud config client）
集中式管理分布式环境中的配置信息
基于 spring 环境提供配置管理，与 spring 系列框架无缝结合
可用于任何语言开发环境
默认基于 GIT 仓库实现版本控制

2 分布式配置中心结构图



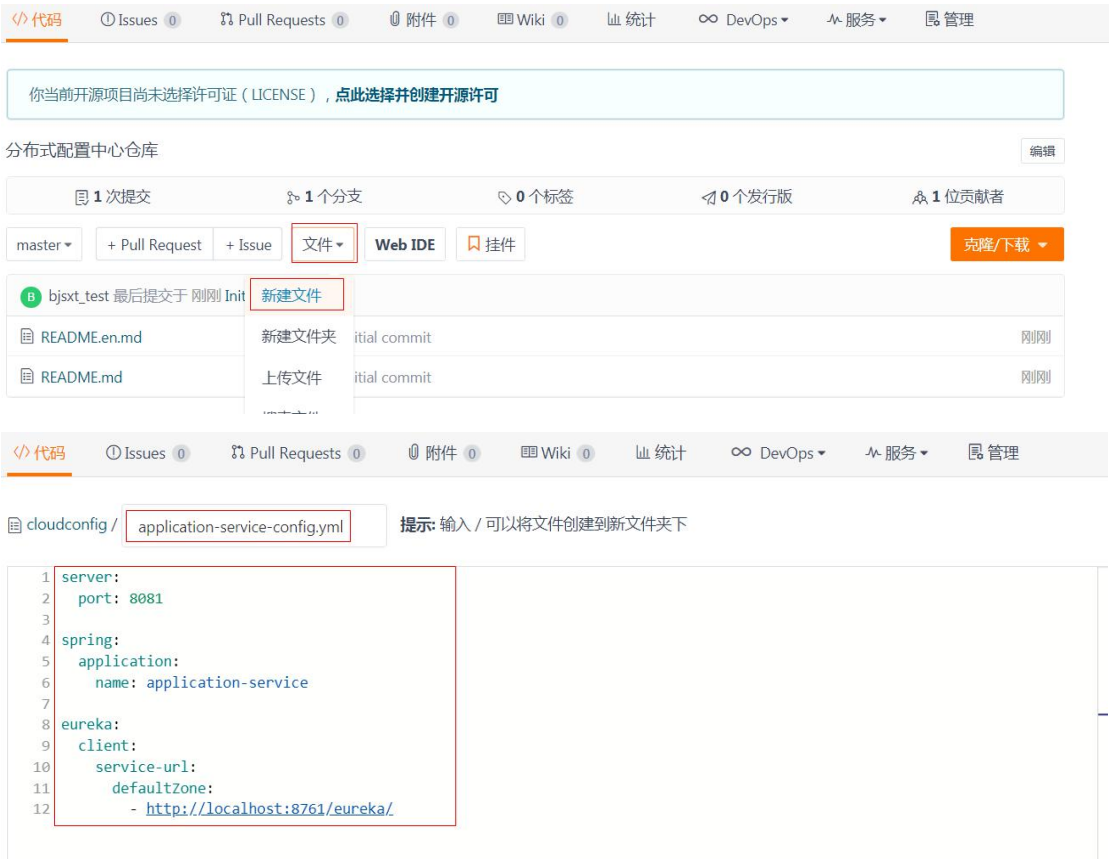
二、 基于 Gitee 管理配置文件

在 Gitee 中集中管理配置文件，为保证文件集中化管理，独立创建仓库。具体过程如下：

1 创建仓库

2 创建文件并编辑内容

2.1 application-service-config.yml 配置文件



2.2 application-service-config-dev.yml 配置文件

三、 搭建分布式配置中心服务端

1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>configserver</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
```

```
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-config-server</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
    </dependencies>
</project>
```

2 配置文件 application.yml

```
spring:
  application:
    name: config-server
  cloud: # 配置 Spring Cloud Config 服务端相关信息
    config:
      server:
        git:
          uri: https://gitee.com/bjsxt_test/cloudconfig.git  # 访问的远程仓库
uri
          username: bjsxt_test  # Gitee 用户名
          password: enter_your_password  # Gitee 密码

server:
  port: 8888  # Spring Cloud Config 服务端默认端口为 8888，可以自定义修改。此端
口为 Spring Cloud Config 客户端默认访问端口。
```

3 启动类

```
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

/**
 * @EnableConfigServer - 开启配置中心服务端。
```

```
 *  应用会根据全局配置文件访问GIT远程仓库,并将远程仓库中的配置内容下载到本地。
 */
@SpringBootApplication
@EnableConfigServer
@EnableEurekaClient
public class ConfigServerApp {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApp.class, args);
    }
}
```

4 浏览器测试获取 Gitee 中的配置文件

当配置中心服务端启动后，可以通过浏览器访问配置中心服务端应用，获取我们保存在 Gitee 中的配置文件内容，具体访问路径格式如下：

http://config-server-ip:port/name/profile/label

name - Gitee 中保存的配置文件核心命名

profile - Gitee 中保存的配置文件扩展环境命名

label - Gitee 中的仓库分支

如访问： http://localhost:8888/application-service-config/default/master

代表在 Gitee 中 master 分支下，查找命名为 application-service-config.yml | application-service-config.properties 配置文件。

如访问： http://localhost:8888/application-service-config/dev/master

代表在 Gitee 中 master 分支下，查找命名为 application-service-config-dev.yml | application-service-config-dev.properties 配置文件。

四、 搭建分布式配置中心客户端

每个 Spring Cloud 微服务应用（除 Eureka 和 Config 配置中心服务端）都可以看做是 Spring Cloud Config 客户端应用，都可以通过 Spring Cloud Config 服务端动态获取保存在 Gitee 中的配置文件。

1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationservice</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-config</artifactId>
        </dependency>
    </dependencies>
</project>
```

2 配置文件 bootstrap.yml

使用优先级最高的全局配置文件 bootstrap.properties 来配置当前应用启动时访问哪一个分布式配置中心服务端获取配置文件。

```
spring:
  cloud:
    config:
      uri: http://localhost:8888/  # 分布式配置中心服务端访问地址，默认为
http://localhost:8888/
      name: application-service-config  # 要读取的配置文件主体命名
      profile: default  # 要读取的配置文件扩展环境名， 默认 default
      label: master  # 在 Gitee 仓库的哪一个分支中读取配置文件， 默认 null， 即
在 master 分支中读取
```

五、 配置热刷新

通过 spring-boot-starter-actuator 可以实现配置中心客户端应用不重启，当某事件触发时，重新访问配置中心服务端，获取新的配置文件，并重新初始化本地环境。这种不重启应用实现环境刷新的方式称为热刷新。

具体实现过程如下：

1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloudhystrix</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationservice</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-config</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
    </dependencies>
</project>
```

2 配置文件

```yaml
spring:
  cloud:
    config:
      uri: http://localhost:8888/  # 分布式配置中心服务端访问地址，默认为 http://localhost:8888/
      name: application-service-config  # 要读取的配置文件主体命名
      profile: default  # 要读取的配置文件扩展环境名， 默认 default
      label: master  # 在 Gitee 仓库的哪一个分支中读取配置文件， 默认 null， 即在 master 分支中读取

management:
  endpoints:
    web:
      exposure:
        include:
          - refresh # 开启热刷新服务， 也可以在 Gitee 中的配置文件内定义。
          - info
          - health
```

3 测试热刷新环境

发送 POST 请求到 http://ip:port/actuator/refresh，应用会重新访问配置中心服务端，加载最新的配置文件内容，并刷新当前应用环境。

4 代码中使用远程配置内容

如果代码中使用了 Gitee 中保存的配置文件内容，那么希望热刷新的时候，重新初始化对象属性值，则需要在对应类型上增加注解@RefreshScope，通知 Spring 容器，当前类型的对象，在环境刷新时，需要重新初始化对象。

```java
package com.bjsxt.controller;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.cloud.context.config.annotation.RefreshScope;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RefreshScope
```

```java
public class ServiceController {
    @Value("${test.str}")
    private String str;

    @PostMapping("/testPost")
    public Object testPost(){
        System.out.println("testPost method run");
        return "写操作返回";
    }

    @GetMapping("/testGet")
    public Object testGet(){
        System.out.println("testGet method run");
        return "读操作返回";
    }

    @GetMapping
    public Object first(){
        System.out.println("run");
        try {
            // 用于模拟远程服务调用延时。
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return "测试Spring Cloud Netflix Ribbon开发服务提供者";
    }
}
```